Lab 2 Addendum, Part B: AES-GCM Student-Data Export

ICS570 Cybersecurity Essentials. Group 3: Henry Nii-Amarh Olletey, Kobina Adzeyena Opei Junior, Eric Sunu and Clifford Yeboah. 17 October 2026. Submitted under the Lab 2 assignment once Part B is released.

1. Demonstration

`npm run partb` (`part-b/aes-gcm-demo.js`) encrypts a fictional S001 record with AES-256-GCM using Node's built-in crypto library (OpenSSL); no AES is implemented by hand. A fixed label is bound to the ciphertext as associated data. Output from a group member's Mac (`evidence/partb-output.txt`):

```
Key: 256-bit, random, in memory only (SHA-256 fingerprint 5c0d41513aad...)

Plaintext record: {"studentId":"S001","name":"Ama Mensah","course":"C101","grade":"B+","note":"synthetic
record"}

1. Encrypt and decrypt
   nonce      ea25a29505ad40660e041e71 (96 bits)
   ciphertext 2decb7247ba13ad8ad2710548e21ebf5... (95 bytes)
   tag        857eedda01e91d6dfa2ffbe67e96cff7 (128 bits)
   decrypted  {"studentId":"S001","name":"Ama Mensah","course":"C101","grade":"B+","note":"synthetic record"}
   identical to original: true

2. Fresh nonce per encryption (same key, same record)
   nonce #1 ea25a29505ad40660e041e71   ciphertext #1 2decb7247ba13ad8ad271054...
   nonce #2 347315fae9a38d89298c172f   ciphertext #2 637689ebf1ee88d6ce4a78bb...
   nonces differ: true; ciphertexts differ: true

3. Tamper tests (local copies)
   ciphertext byte 5 flipped: REJECTED, Unsupported state or unable to authenticate data
   tag byte 0 flipped       : REJECTED, Unsupported state or unable to authenticate data
   wrong key                : REJECTED, Unsupported state or unable to authenticate data

All tamper attempts rejected: true
```

2. Results

Check	Result
1. Encrypt, then decrypt with the same key	Decrypted record identical to the original
2. Encrypt the same record twice	New random 96-bit nonce each time; nonces and ciphertexts differ
3a. Flip one ciphertext byte in a local copy	Rejected: unable to authenticate data
3b. Flip one tag byte in a local copy	Rejected: unable to authenticate data
3c. Decrypt with a different key	Rejected

3. Key protection and nonce uniqueness

The 256-bit key is generated randomly, held only in memory and shown only as a fingerprint. In a real export it would live in a key store or come from an environment secret, never in the code or next to the file. Each encryption draws a fresh random 96-bit nonce, stored beside the ciphertext. Reusing a nonce under the same key repeats GCM's keystream, so XOR-ing two ciphertexts gives the XOR of the two plaintexts. It also exposes the authentication key, which lets an attacker forge valid tags.

4. Why encryption does not replace authentication or authorization

AES-GCM protects the exported file's confidentiality and integrity, but only against people who do not hold the key. It cannot say who is asking for the data; that is the passkey's job. Nor can it say whether that person may see this record; that is the policy's job. If the portal wrongly let someone through, the server would decrypt the data for them. Each control answers a different question, so each has to be designed and tested on its own.